

Guess Market - Exercise 3

Name: Dan Alushe

ID: 212472864

Email: danalushe15@gmail.com

GitHub: <https://github.com/danal15/guess-market>

Bonus implemented

Chat. Every logged in user can open the Chat panel at the bottom of the window and talk to everyone else. It all goes through the server: the messages are kept there and arrive with the ordinary refresh, so everyone sees what everyone writes. Only the messages you have not seen yet are sent each time, not the whole conversation.

How to run

- ☐ Put guess-market.war into tomcat\webapps and start Tomcat. It deploys by itself under /guess-market.
- ☐ Then go into the ex3-client folder and double click run.bat (or run: java -jar guess-market-client.jar). Start it as many times as you like - each one is a separate user.
- ☐ The client already knows where the server is (<http://localhost:8080/guess-market>), so there is nothing to set up. If you ever need another address, pass it as an argument to run.bat.
- ☐ Tomcat 10.1 and Java 25. The servlets use jakarta.servlet, which is what Tomcat 10 and up provide.
- ☐ JavaFX and Gson are packed inside the client jar, and Gson is inside the war, so nothing has to be installed.
- ☐ Log in with any name. There are no passwords. A name already in use is refused and you can try another.
- ☐ A new user starts with \$0. Use "Load funds..." on the Account tab to put money in - you need money before you can open an event or buy anything.
- ☐ Upload an events file with "Upload Events File..." on the Account tab. Exercise 3 files only. You become the market maker of every event in the file you upload.

How to use it

- ☐ Events tab - view only. Every event in the system, from everybody's files, with filters by method / status / commission (each one has "All"). Clicking an event shows its details: for LMSR the prices, account, trade history and chart; for an order book both books, the statistics, the participants and the chart.
- ☐ Account tab - your own screen, and where the actions are. Your balance and the "Load funds" button, every movement of money in your account on its own line, the list of other users, the upload button, and the events with what you are allowed to do in each.
- ☐ Of other users you only see their name, their balance and whether they are a market maker. Their account lines are private, the same as yours. The order books are public though, so you do see who placed which order - that is how an order book works.

- Everything refreshes by itself once a second, so other people's uploads, trades and closings appear without you pressing anything.

Project structure

Four modules. engine and desktop are from before and were kept; server and client are new.

engine - the system itself, with no JavaFX and no servlet code in it. It is the passive module: it answers whoever calls it and knows nothing about them, which is exactly why the same engine could move from behind a window in exercise 2 to behind a web server here. It now also holds the logged in users, the money movements and the chat.

server - the war. Thin servlets that do nothing but call the engine and turn its answer into JSON. EngineHolder creates the one engine the whole application shares, GmServlet holds what every endpoint needs (reaching the engine, knowing who is asking, writing JSON, turning a refusal into a message a person can read), and then one small servlet per thing you can do: login, sync, upload, funds, event open and close, trade, event/new, chat.

client - the new JavaFX application. MarketClient implements the same GMEngine interface the screens were already written against, only now every call goes to the server over HTTP. That is what let the exercise 2 screens be reused untouched. Transport carries the calls, keeps the session cookie, and rebuilds the engine's own exceptions out of the server's answers, so the screens keep catching exactly what they always caught.

desktop - the exercise 2 application. The client reuses its event details panel, involvement panel, trade forms, balance chart, create-event dialog, dialogs, tables, skins and stylesheets as they are.

Main new classes

SnapshotDTO - everything one user's screen shows, gathered in a single pass over the market. The clients poll, and a screen that asked for each piece separately could catch the market in the middle of a trade and show a balance from before it beside holdings from after it. Taking it all at once means what you see always describes one moment, and the refresh is one request instead of six.

Movement and MovementDTO - one movement of money: what it was for, which event it belonged to, the amount, and the balance left after it. This is what the account list shows, one line per movement.

EngineHolder - a servlet context listener that creates the single engine when the war deploys and hands it to every servlet. It has to be exactly one, or users who logged in to one engine would be invisible to the other.

GmServlet - the base class the endpoints share. A subclass writes the one line that calls the engine; JSON, errors and who is asking all live here.

Transport and MarketClient - the client side of the conversation. MarketClient is the engine as far as the screens are concerned; Transport is the part that actually speaks HTTP.

ClientRootController - holds the two tabs together and runs the once-a-second refresh, off the interface thread and one request at a time.

AccountTabController - the user's own screen: balance, loading funds, the movement list, the other users, uploading a file, and the actions available on the selected event.

MarketFileLoader.loadInto - reads an exercise 3 file and adds it to a market that is already running. The exercise 2 reader is still there beside it, and they share all the parsing.

Decisions I made

- ☐ A new user starts with a balance of \$0. Exercise 2 read the starting money out of the file, and exercise 3 files have no users in them at all - but it is the same exercise that adds "load funds", so starting at nothing and loading what you want is what that is for.
- ☐ Exercise 2 said a user whose balance goes below zero is blocked from then on. I kept that, but loading funds now lifts the block once the account is covered again, otherwise the new ability to load funds would be useless to the one person who needs it. Money arriving any other way - a payout, a commission - does not lift it. Only a deliberate deposit does.
- ☐ Events are identified by name. The exercise 3 schema has no id, and a name that already exists anywhere in the system is refused. Names are compared ignoring case and surrounding spaces, so the same event cannot arrive twice wearing a hat. Internally an event still carries a number, handed out by the server; the client never invents one.
- ☐ A file is taken in whole or not at all. Everything is checked - including that no name collides - before a single event is added. Other people are reading this market at the same time, and half a file would be worse than none of it.
- ☐ Two events with the same name inside one file are refused as well, not only a collision with something already loaded.
- ☐ The file checks are the ones from exercise 1 (it is an .xml file, the commission is between 0 and 90) plus the new name check, not the ones from exercise 2 - exercise 2's extra checks were all about the users section, which exercise 3 files do not have. A file that does have a users section is refused with a message saying it looks like an exercise 2 file.
- ☐ The file is never written to disk on the server. The client reads the bytes and posts them, and the server parses them straight out of the request.
- ☐ Events have exactly two options. The exercise 3 schema allows at most two, both sample files have two, the sketch shows two order books, and appendix A says it is looking at the two-option case only. A file with any other number is refused with a clear message.
- ☐ Who you are comes from the session, never from the request, so a client cannot ask to act on somebody else's behalf.
- ☐ The refresh is once a second, inside the two seconds allowed. The whole picture comes back each time except the chat, where only what you have not seen is sent.
- ☐ Only one request is in flight at a time. If an answer is slow the next tick is skipped rather than piling up behind it.
- ☐ Every public method of the engine is synchronised. Several requests arrive at once now, and almost every action is a transaction over several objects at once: take the money, move the shares, update the counters, record the trade. Guarding each field on its own would have been more code and still wrong.
- ☐ The filters are applied in the client. One refresh brings the whole market, and the filtering rule itself lives on EventFilterDTO so the engine and the client use the same one instead of two copies.
- ☐ If the server goes away the client says so quietly in the top bar and keeps trying, instead of opening a dialog every second.

- Nothing is saved anywhere. When Tomcat stops the market is gone, which is what the exercise asks for. The save and load from exercise 1 is still in the engine but is not reachable from the client.
- Everything else is unchanged from exercise 2: the LMSR maths, the order book matching and minting, the commissions, the rule that a price cannot exceed d minus a cent, and closing an event and paying the winners.